Lab Manual 13

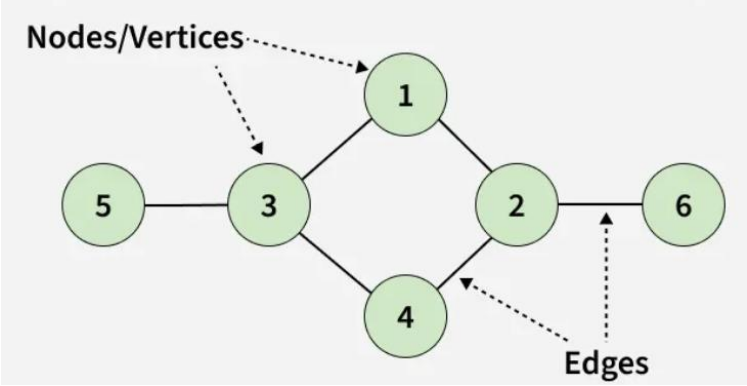
Objectives:

- Understand graph basics and the adjacency list representation.
- Implement Depth First Search (DFS) using an adjacency list and its applications.

In the world of data structures, Graphs are powerful tools for modeling relationships between objects. They are widely used in networking, social media connections, route planning, dependency resolution, and many other real-world problems. Unlike linear data structures like arrays, stacks, or queues, graphs allow representation of complex, non-linear relationships between elements (nodes).

The Need for Graphs

While arrays, stacks, and queues manage data sequentially, they cannot naturally represent relationships like "friendship," "road between cities," or "task dependencies." Graphs provide a flexible way to model these connections efficiently



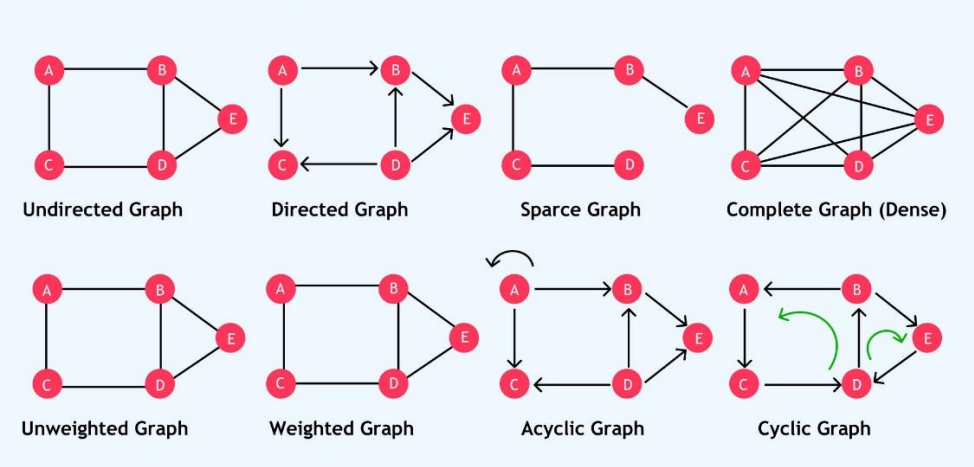
Components of Graph

- Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.
- Edges are drawn or used to connect two nodes of the graph. Every edge can be labelled/unlabelled.

Types of Graphs

Graphs can be:

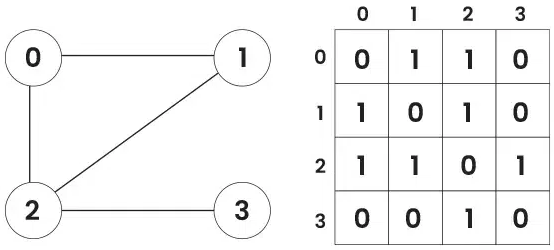
- Directed / Undirected: Edges have direction (or not).
- Weighted / Unweighted: Edges may carry a cost or weight.
- Cyclic / Acyclic: Graphs may contain cycles.



Graph Representation
Graphs can be represented in two common ways:

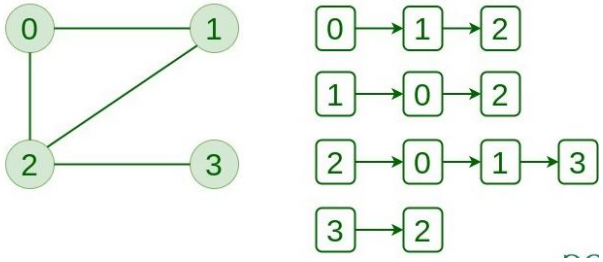
Representation	Description	Space Complexity
Adjacency Matrix	A 2D array of size V×V where matrix[i][j] = 1 (or weight) if edge exists	$O(V^2)$
Adjacency List	An array of lists where each index represents a vertex and stores connected vertices	O(V + E)

Adjacency Matrix of Graph



	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

Adjacency List of Graph



Depth First Search (DFS)

Depth First Search (DFS) is a fundamental graph traversal algorithm used to explore nodes and edges of a graph. It goes as deep as possible along each branch before backtracking.

Pseudocode

DFS can be implemented using recursion or an explicit stack:

1. Start from a source vertex.
2. Mark it as visited.
3. Recursively visit all unvisited neighbors (or push them onto a stack).
4. Repeat until all reachable nodes are visited.

Time Complexity: O(V + E) for adjacency list, $O(V^2)$ for adjacency matrix.
Space Complexity: O(V) for visited array + recursion stack.

Advantages of DFS

- Simple and easy to implement
- Uses less memory for sparse graphs (with adjacency list)
- Can be modified for advanced algorithms like cycle detection, topological sorting

Limitations of DFS

- Can get trapped in deep paths if graph is very large or infinite
- Not guaranteed to find the shortest path (use BFS for shortest paths in unweighted graphs)

Applications of DFS

- Detecting cycles in a graph
- Finding connected components
- Topological sorting of a DAG
- Pathfinding in mazes and puzzles

Background

In the world of social networking, every person is connected to others through friendships, just like nodes in a graph. Imagine **Facebook Friends Network**, where each user is a vertex, and friendships are edges connecting them. Understanding this network is crucial for recommending friends, analyzing communities, or detecting influential people.

Professor Zuckerberg, a visionary social data scientist, has tasked you, a **Graph Engineer**, with simulating a mini social network. Your job is to represent friendships, explore connections, and analyze the network using **Depth First Search (DFS)**. Unlike simple lists or arrays, graphs allow modeling complex relationships, making it easy to traverse and study the network structure.

In-Lab Tasks

STL containers (like queue and stack) are allowed if needed.

You are required to implement a **Graph class** in C++, representing a social network of friends.
Each user (vertex) has attributes:

- **id**: unique user identifier
- **name**: user's name (e.g., Alice, Bob, Charlie)

Your **Graph** class should support the following **functions via a menu**:

1. **Add User** – Add a new user to the network.
2. **Add Friendship** – Create a connection between two users.
3. **Display Friends** – Show all friends of a given user.
4. **DFS Traversal** – Starting from a user, traverse all reachable friends using DFS.
5. **Count Friend Circles** – Count the number of connected groups in the network.
6. **Find Popular Users** – Display users with the maximum number of friends.
7. **Detect Isolated Users** – Show users with no friends.
8. **Mutual Friends** – Display the list of common friends between two users
9. **Suggest Friends** – Recommend friends-of-friends who are not yet direct friends.
10. **Remove User** – Delete a user and all associated friendships.
11. **Exit** – Terminate the simulation.

Submission Guidelines

1. submit following the only file strictly following the naming convention
    a. l241234.cpp
